AI Engineer – Module 3

Bài 1: Tổng quan GenAI/LLM, kiến trúc Transformer và các mô hình nền tảng

Lecture – 2,5 giờ

Tư duy xuyên suốt: **Nghiệp vụ** → **Dữ liệu** → **Loại AI** → **Công cụ** → **Đánh giá** → **Rủi ro** → **Quyết định**

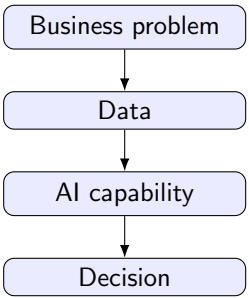
1. Bắt đầu từ nghiệp vụ, không bắt đầu từ model

Câu hỏi sai

"Model AI nào đang mạnh nhất?"

Câu hỏi đúng hơn

- Quyết định/hành động nghiệp vụ nào cần hỗ trợ?
- Dữ liệu đầu vào là gì?
- Nếu AI sai thì hậu quả là gì?



2. Mục tiêu đầu ra của Bài 1

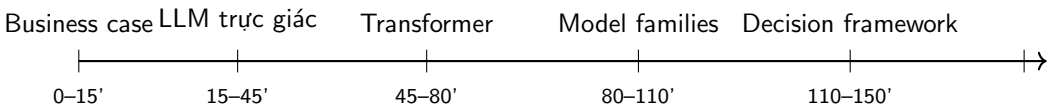
Sau buổi học, học viên có thể:

1. Phân biệt ở mức sử dụng: Rule, ML/NLP, CV, Generative AI, RAG, Agent.
2. Giải thích LLM bằng trực giác: **token – context – next-token prediction – attention**.
3. Hiểu kiến trúc Transformer ở cấp khái niệm: **Self-Attention** và **Multi-Head Attention**.
4. So sánh ba họ mô hình nền tảng **BERT, GPT, LLaMA** – điểm khác biệt và ứng dụng.
5. Dùng khung quyết định để trả lời: **có nên dùng LLM cho use case này không?**

Không phải mục tiêu

Không chứng minh công thức Transformer, không tự cài đặt attention, không train LLM từ đầu.

3. Bản đồ 150 phút lecture



Nguyên tắc lớp học

Mỗi khái niệm mới đều đi kèm một ví dụ nghiệp vụ và ít nhất một câu hỏi về failure mode / human control.

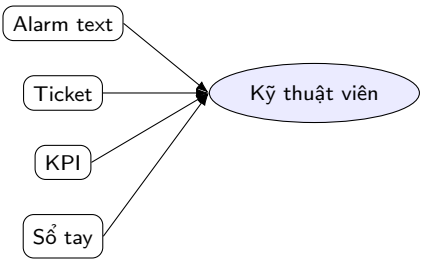
4. Case mở đầu: incident mạng

Tình huống

Site `HN-023` mất kết nối chập chờn. Trong 3 giờ có 15 alarm, 2 ticket, một số KPI bất thường và một đoạn mô tả của kỹ thuật viên.

Kỹ thuật viên cần:

- hiểu nhanh chuyện gì đang xảy ra;
- ưu tiên bước kiểm tra tiếp theo;
- tránh bỏ sót dấu hiệu quan trọng;
- không tự động thay đổi cấu hình nếu chưa đủ căn cứ.



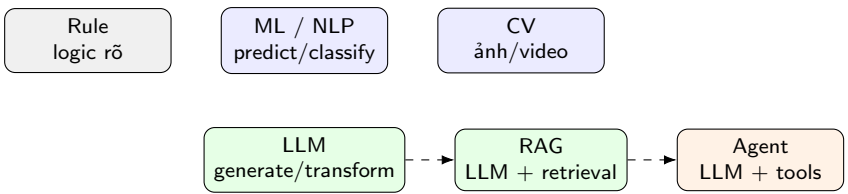
5. Sáu tình huống nghiệp vụ

1. Dự báo KPI ngày mai.
2. Phân loại ticket vào nhóm sự cố.
3. Tóm tắt báo cáo incident dài.
4. Hỏi–đáp từ sổ tay vận hành nội bộ.
5. Đọc ảnh thiết bị để phát hiện đối tượng/lỗi.
6. Đề xuất bước kiểm tra tiếp theo theo quy trình.

Câu hỏi cho lớp

Với mỗi tình huống, bạn sẽ nghĩ tới: **Rule / ML-NLP / CV / LLM / RAG / Agent / Không dùng AI**?

6. Bản đồ công cụ AI ban đầu



Không có "công cụ tốt nhất". Chỉ có công cụ phù hợp hơn với loại dữ liệu, mục tiêu, chi phí và rủi ro.

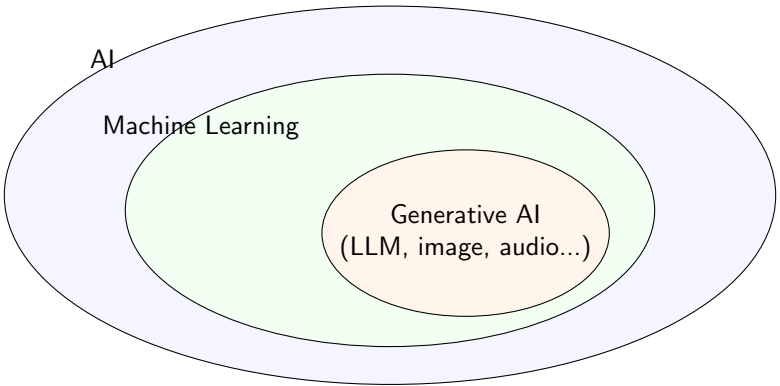
7. Quick decision challenge

Bài toán	**Ứng viên đầu tiên**
Kiểm tra mã ticket đúng format?	Rule / validation
Dự báo lưu lượng giờ tới?	ML / time-series
Nhận diện vật thể trong ảnh?	CV
Tóm tắt incident tự do?	LLM
Hỏi từ tài liệu nội bộ cập nhật?	RAG
Gọi nhiều công cụ theo workflow?	Agent

Điểm cần nhớ

Nếu rule/process đơn giản giải quyết đủ tốt, **không cần ép bài toán thành AI**.

8. AI, Machine Learning và Generative AI



9. Rule-based: khi logic đã rõ

Ví dụ

Nếu `severity = critical` và `site_type = core` $\Rightarrow$ tạo cảnh báo ưu tiên cao.

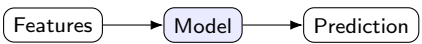
- Dễ giải thích, dễ kiểm thử, chi phí thấp.
- Phù hợp khi điều kiện ít thay đổi và có thể mô tả rõ.
- Không cần "AI" chỉ để thay thế một vài câu `if/else`.

Giới hạn

Khi dữ liệu mơ hồ, ngôn ngữ tự nhiên đa dạng hoặc số quy tắc bùng nổ, rule trở nên khó duy trì.

10. Machine Learning truyền thống: học từ ví dụ

- Input thường được biểu diễn thành features.
- Model học quan hệ giữa input và target từ dữ liệu đã có nhãn.
- Mạnh ở prediction/classification khi bài toán ổn định và dữ liệu đủ tốt.



Ví dụ: dự báo KPI, churn, anomaly score, ticket routing.

11. NLP: biến văn bản thành dữ liệu có thể xử lý

Các nhóm tác vụ phổ biến:

- Text classification
- Named Entity Recognition
- Information extraction
- Sentiment
- Semantic similarity
- Question answering

Ví dụ nghiệp vụ

Ticket: "Site HN-023 mất kết nối từ 09:20" → trích `site_id`, `symptom`, `time`.

12. Computer Vision: khi đầu vào là ảnh/video

- Classification: ảnh thuộc lớp nào?
- Detection: vật thể nào ở đâu?
- Segmentation: vùng pixel nào thuộc đối tượng?
- OCR: chữ gì xuất hiện trong ảnh?

Ví dụ

Ảnh trạm/hạ tầng $\rightarrow$ phát hiện thiết bị, vật thể lạ, biển báo, vùng hư hỏng.

Bài 4 sẽ đi sâu ở mức ứng dụng và đọc kết quả model pre-trained.

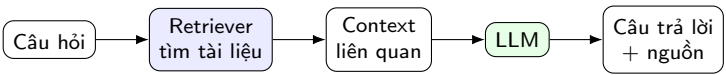
13. Generative AI: sinh đầu ra mới

- Sinh văn bản, mã nguồn, hình ảnh, âm thanh hoặc dữ liệu có cấu trúc.
- Với LLM, input thường là ngôn ngữ tự nhiên + context.
- Điểm mạnh: linh hoạt, có thể xử lý nhiều task bằng cùng một model.

Ví dụ

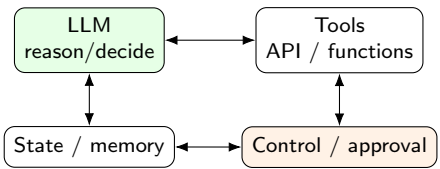
Incident dài → tóm tắt; ticket tự do → JSON; log + mô tả → gợi ý bước kiểm tra.

14. RAG: khi LLM cần tri thức nội bộ



RAG không làm model "thông minh hơn"; nó đưa **bằng chứng phù hợp vào context** trước khi model trả lời. (Bài 5–6)

15. Agent: khi AI cần dùng công cụ theo workflow



Agent không đồng nghĩa "AI tự chủ hoàn toàn". Bài 8 sẽ tập trung vào tool-use và human approval.

16. LLM nhìn thấy gì? Text trước hết thành token



Trực giác

Model không đọc nguyên “từ” theo đúng trực giác con người; nó xử lý một chuỗi đơn vị số hóa gọi là **token**.

17. Token không nhất thiết là một từ

- Có thể là một từ, một phần từ, dấu câu hoặc chuỗi ký tự.
- Từ chuyên ngành, mã thiết bị, URL hoặc code có thể bị tách thành nhiều token.
- Cùng một nội dung, ngôn ngữ khác nhau có thể dùng số token khác nhau.

Ví dụ khái niệm

`HN-023` có thể được biểu diễn bởi nhiều token nhỏ hơn thay vì một token duy nhất.

Không cần học thuật toán tokenizer chi tiết trong Module 3.

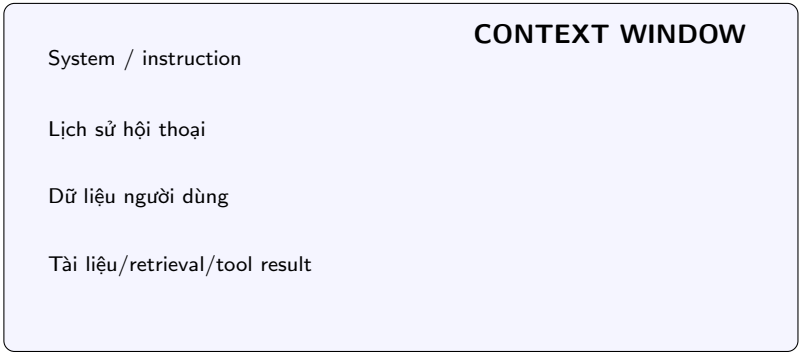
18. Tại sao token quan trọng trong ứng dụng?

1. **Context limit**: model chỉ xử lý hữu hạn token trong một lần gọi.
2. **Latency**: input/output dài thường chậm hơn.
3. **Cost**: nhiều API tính chi phí theo lượng token.
4. **Thiết kế prompt/RAG**: phải chọn thông tin nào thật sự cần đưa vào context.

Thông điệp

"Đưa toàn bộ dữ liệu vào prompt" không phải chiến lược có thể mở rộng vô hạn.

19. Context: những gì model đang được phép nhìn thấy



Context không phải "trí nhớ vô hạn". Những gì nằm ngoài context hiện tại có thể không được model sử dụng.

20. Context đến từ đâu?

Nguồn	Vai trò
System instruction /	Quy định vai trò, phạm vi, nguyên tắc trả lời
User input	Nhiệm vụ và dữ liệu người dùng cung cấp
Conversation history	Ngữ cảnh các lượt trước
Retrieved documents	Bằng chứng bổ sung từ kho tri thức
Tool results	Kết quả API/hàm/công cụ được gọi

Câu hỏi

Nếu một thông tin quan trọng không có trong pre-training và cũng không nằm trong context, model lấy nó từ đâu?

21. Context window có giới hạn



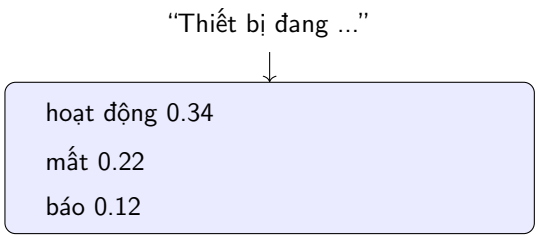
Overflow → cần chọn/chia/retrieve

- Context quá ngắn: thiếu thông tin.
- Context quá dài: tốn chi phí, có thể làm thông tin quan trọng bị "chìm".
- RAG là một cách đưa **đúng phần liên quan** vào context thay vì đưa tất cả.

22. Cốt lõi: dự đoán token tiếp theo

Trực giác

Cho một chuỗi token hiện tại, model tính khả năng của nhiều token có thể xuất hiện tiếp theo.



Đây chỉ là minh họa trực giác, không phải xác suất thật của một model cụ thể.

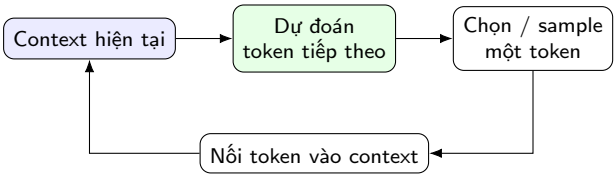
23. Xác suất không đồng nghĩa sự thật

- Token "có xác suất cao" nghĩa là phù hợp theo mô hình thống kê của model.
- Điều đó **không đảm bảo** mệnh đề tạo ra đúng với thực tế.
- Vì vậy câu văn trôi chảy có thể vẫn chứa thông tin sai hoặc không có bằng chứng.

Đây là gốc của một số hallucination

Model tối ưu cho việc sinh chuỗi phù hợp với context, không phải cơ chế kiểm chứng chân lý tự động.

24. Generation là một vòng lặp autoregressive

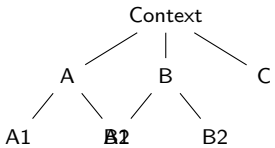


Lặp nhiều lần $\Rightarrow$ một đoạn văn, JSON,

code hoặc câu trả lời hoàn chỉnh.

25. Tại sao cùng prompt có thể ra kết quả khác?

- Có nhiều token tiếp theo đều hợp lý.
- Cơ chế sampling có thể chọn các nhánh khác nhau.
- Một thay đổi nhỏ ở prompt/context có thể dẫn tới chuỗi sinh khác.



Hệ quả

Đầu ra LLM cần được **đánh giá trên test set**, không chỉ xem một lần demo đẹp.

26. Temperature: điều chỉnh mức ngẫu nhiên

Temperature thấp

- lựa chọn tập trung hơn;
- thường ổn định hơn;
- phù hợp task cần format/chính xác hơn.

Temperature cao

- phân phối lựa chọn rộng hơn;
- đa dạng hơn;
- có thể tăng biến thiên đầu ra.

Không hiểu nhầm

Temperature **không bổ sung kiến thức mới** và không phải "nút tăng độ thông minh".

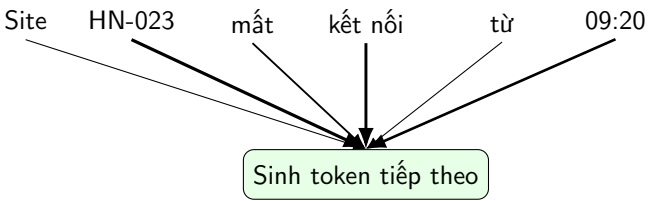
27. Vì sao cần Transformer?

- Các mô hình tuần tự cũ (RNN/LSTM) xử lý token lần lượt, khó nắm quan hệ xa và khó song song hóa.
- Transformer (2017) thay cơ chế tuần tự bằng **attention**: mỗi token có thể "nhìn" trực tiếp mọi token khác trong context.
- Nhờ vậy: huấn luyện song song tốt hơn, nắm được phụ thuộc xa, mở rộng tới mô hình rất lớn.

Trong Bài 1

Ta hiểu Transformer ở **cấp khái niệm** để dùng đúng, không chứng minh công thức.

28. Attention: trực giác "đang chú ý vào phần nào?"



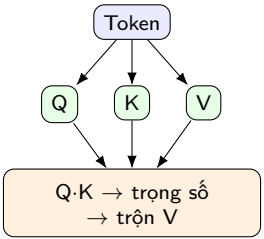
Attention giúp model kết hợp các phần của context với mức độ liên quan khác nhau (độ dày nét = trọng số).

29. Self-Attention: Query, Key, Value

Với mỗi token, model tạo ba vector:

- **Query (Q)**: "tôi đang tìm thông tin gì?"
- **Key (K)**: "tôi chứa thông tin gì?"
- **Value (V)**: "nội dung tôi mang theo".

Điểm liên quan = độ khớp giữa Q của token này với K của các token khác; dùng điểm đó để **trộn có trọng số** các V.



"Self": Q, K, V đều lấy từ chính chuỗi đầu vào, nên token học quan hệ với các token khác trong cùng câu/đoạn.

30. Self-Attention trong case incident

Câu hỏi: *"Site nào mất kết nối và từ thời điểm nào?"*

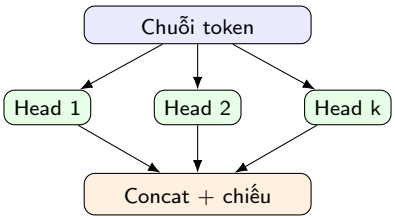
Context

"09:10 có cảnh báo nhiệt độ tại DN-011. 09:20 site HN-023 mất kết nối. 09:45 kỹ thuật viên bắt đầu kiểm tra."

- Query cho "site nào / thời điểm nào" khớp mạnh với Key ở `HN-023` và `09:20`.
- Attention là cơ chế cho phép các token liên hệ với các token khác trong context.
- Nhưng attention tốt không tự động đảm bảo thông tin đầu vào là đúng.

31. Multi-Head Attention

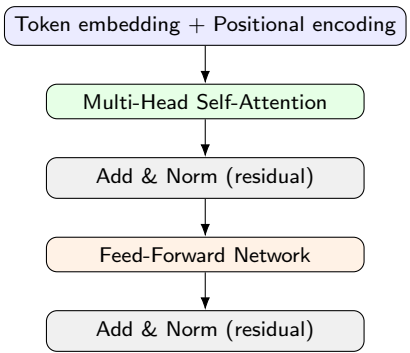
- Thay vì một "góc chú ý", model dùng **nhiều head** song song.
- Mỗi head có thể học một loại quan hệ khác nhau: vị trí/thời gian, thực thể, cú pháp, nguyên nhân–kết quả...
- Kết quả các head được ghép lại rồi chiếu về một biểu diễn chung.



Trực giác

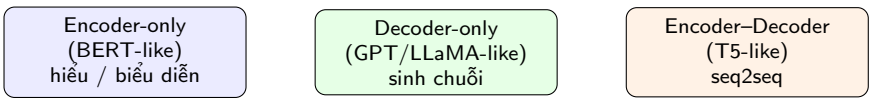
Giống nhiều chuyên gia cùng đọc một đoạn, mỗi người chú ý một khía cạnh, rồi tổng hợp lại.

32. Một khối Transformer gồm gì?



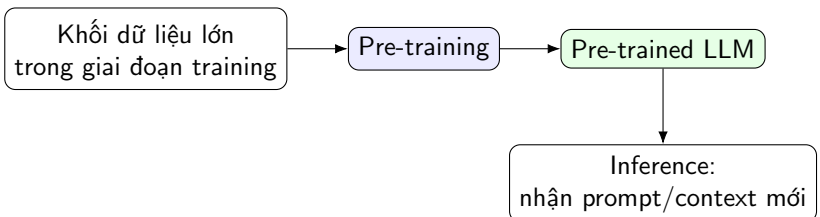
Positional encoding: vì attention không có thứ tự tự nhiên, cần thêm thông tin vị trí. Nhiều khối xếp chồng tạo thành mô hình sâu.

33. Ba kiểu kiến trúc Transformer



- Encoder-only: đọc toàn bộ context hai chiều, mạnh cho hiểu/phân loại/trích xuất.
- Decoder-only: sinh token tuần tự, mạnh cho generation/chat.
- Encoder–Decoder: ánh xạ chuỗi vào $\rightarrow$ chuỗi ra (dịch, tóm tắt theo khuôn).

34. Pre-training: model học trước khi ta dùng



Trong Module 3, ta ưu tiên **pre-trained first**: dùng model có sẵn thay vì huấn luyện mô hình lớn từ đầu.

35. Inference khác training

Training / fine-tuning	**Inference**
Cập nhật tham số model	Không cập nhật trọng số
Cần dữ liệu và tài nguyên huấn luyện	Dùng model để xử lý input mới
Có thể kéo dài	Theo request / batch
Mục tiêu: học hành vi/khả năng	Mục tiêu: tạo prediction/output

Bài 8 sẽ dùng decision framework Prompt vs RAG vs Fine-tuning vs Agent.

36. LLM không phải database và không phải search engine

Database	Search	LLM
Truy xuất dữ liệu đã lưu, có cấu trúc/nguồn rõ.	Tìm tài liệu/trang có liên quan.	Sinh output dựa trên patterns + context.

Hệ quả

Nếu cần câu trả lời có căn cứ từ tài liệu nội bộ, chỉ "hỏi LLM" thường chưa đủ; cần retrieval/grounding.

37. Đừng học “model zoo”: hãy học kiểu năng lực

BERT-like
hiểu / biểu diễn

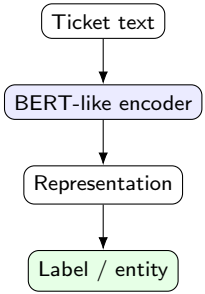
GPT-like
sinh chuỗi

LLaMA / open-weight
triển khai / tùy chỉnh

Mục tiêu: biết **khi nào một kiểu model phù hợp**, không phải thuộc tên hàng chục model/phiên bản.

38. BERT-like: biểu diễn và hiểu văn bản

- Kiến trúc **encoder-only**, đọc ngữ cảnh hai chiều.
- Phù hợp nhiều tác vụ "đọc hiểu" hơn là sinh văn bản dài tự do.
- Có thể tạo biểu diễn văn bản hoặc làm nền cho classifier/NER.



39. Use case BERT-like

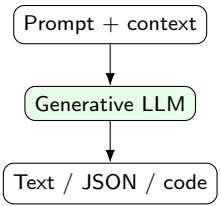
- Ticket classification / routing.
- Sentiment hoặc intent classification.
- NER / information extraction.
- Semantic similarity / embedding-based search.

Khi đáng cân nhắc

Task lặp lại nhiều, nhãn/schema rõ, cần output ổn định và có model chuyên biệt phù hợp.

40. GPT-like: model sinh nội dung

- Kiến trúc **decoder-only**: nhận prompt/context và sinh chuỗi token tiếp theo.
- Linh hoạt cho nhiều nhiệm vụ mà không cần train riêng cho từng task.
- Có thể sinh text, code, JSON hoặc giải thích.



41. Use case GPT-like

- Tóm tắt incident
- Viết nháp báo cáo
- Chuyển text → JSON
- Q&A theo context
- Phân loại nhẹ
- Giải thích log / code

Điều kiện

Tính linh hoạt cao phải đi cùng kiểm thử, structured output, grounding và human review phù hợp rủi ro.

42. LLaMA và các open-weight LLM

- Họ LLM **decoder-only** có trọng số khả dụng theo điều kiện/license cụ thể.
- Có thể phục vụ nhu cầu tự triển khai, fine-tune hoặc kiểm soát hạ tầng tốt hơn.
- "Open" không đồng nghĩa không có ràng buộc license, chi phí hoặc vận hành.

Góc nhìn Module 3

Hiểu lựa chọn **API hosted vs model tự triển khai**; fine-tune nhẹ (LoRA/QLoRA) học ở Bài 7.

43. BERT vs GPT vs LLaMA: điểm khác biệt và ứng dụng

Tiêu chí	**BERT-like**	**GPT-like**	**LLaMA / open-weight**
Kiến trúc	Encoder-only	Decoder-only	Decoder-only
Hướng đọc context	Hai chiều	Trái → phải	Trái → phải
Thế mạnh	Hiểu, phân loại, trích xuất	Sinh, chuyển đổi, hội thoại	Sinh + tự triển khai/tùy chỉnh
Output điển hình	Nhãn, entity, embedding	Text, JSON, code	Text, JSON, code
Triển khai	Model nhỏ, inference rẻ	Thường qua API hosted	Self-host / private deploy
Ứng dụng	Routing ticket, NER, search	Tóm tắt, draft, Q&A	Giống GPT + kiểm soát dữ liệu

44. Hosted API hay tự triển khai model?

Hosted API	**Self-host / private deployment**
Khởi động nhanh	Kiểm soát hạ tầng nhiều hơn
Ít gánh nặng serving	Cần GPU/serving/monitoring
Chi phí theo usage phổ biến	Có chi phí hạ tầng/vận hành
Phải xem chính sách dữ liệu	Có thể phù hợp yêu cầu nội bộ
Phụ thuộc service/provider	Tự chịu trách nhiệm cập nhật

Quyết định thực tế phụ thuộc chính sách, hiệu năng, chi phí và năng lực vận hành của tổ chức.

45. Pre-trained first

Thứ tự ưu tiên thực dụng

1. Dùng model/API có sẵn để kiểm tra use case.
2. Cải thiện prompt / output format / evaluation (Bài 3).
3. Nếu thiếu tri thức nội bộ $\rightarrow$ cân nhắc RAG (Bài 5–6).
4. Nếu cần workflow / tool-use $\rightarrow$ cân nhắc Agent (Bài 8).
5. Fine-tuning (Bài 7) chỉ khi có lý do rõ ràng về hành vi/đặc thù task.

46. Những tác vụ LLM thường làm tốt

Nhóm task	**Ví dụ**
Summarization	Tóm tắt incident / email / báo cáo
Transformation	Rewrite, chuẩn hóa, text $\rightarrow$ JSON
Q&A theo context	Trả lời từ dữ liệu được cung cấp
Light classification	Gán nhãn khi bộ nhãn nhỏ và mô tả rõ
Drafting / coding	Viết nháp, giải thích, hỗ trợ code

"Làm tốt" $\neq$ "luôn đúng"

Mỗi use case vẫn cần test set và error analysis.

47. Task: Summarization

Input

Incident gồm timeline, alarm, hành động đã thực hiện, impact và giả thuyết nguyên nhân.

Output mong muốn

3–5 bullet: symptom, impact, timeline, action, unresolved issue.

Phải kiểm tra:

- có bỏ sót fact quan trọng không?
- có thêm fact không tồn tại không?
- có biến "giả thuyết" thành "kết luận" không?

48. Task: Transformation / Structured output

Ví dụ chuyển mô tả tự do thành schema:

Input

"HN-023 mất kết nối từ 09:20, mức ảnh hưởng cao, chưa rõ nguyên nhân."

Output

```
{site_id: HN-023, symptom: connectivity_loss, severity: high, time: 09:20,
suspected_cause: null}
```

Structured output dễ kiểm tra và tích hợp hệ thống hơn text tự do; Bài 3 sẽ học kỹ hơn.

49. Task: Classification nhẹ

- Cho trước bộ nhãn nhỏ: Connectivity / Performance / Hardware / Configuration / Other.
- Mô tả mỗi nhãn rõ ràng.
- Yêu cầu model chỉ trả một nhãn hoặc JSON chuẩn.

Rủi ro

Nhãn mơ hồ hoặc chồng lấn làm model khó ổn định; lỗi có thể đến từ **label design**, không chỉ từ model.

50. Task: Drafting và hỗ trợ code

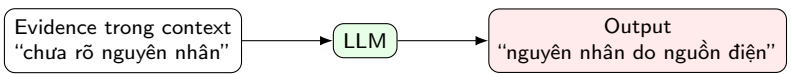
LLM có thể hỗ trợ:

- tạo nháp báo cáo sau sự cố;
- giải thích log / stack trace;
- tạo đoạn code mẫu hoặc test case;
- chuyển yêu cầu tự nhiên thành pseudo-code.

Human-in-the-loop

Code hoặc nội dung nghiệp vụ quan trọng vẫn cần review, test và quyền hạn phù hợp trước khi dùng thật.

51. Failure mode 1: Hallucination



Không có bằng chứng

Hallucination: model tạo thông tin nghe hợp lý nhưng không được hỗ trợ bởi dữ liệu/context đáng tin cậy.

52. Failure mode 2: Prompt sensitivity

Prompt A

"Phân loại ticket này."

Output có thể không đúng format, giải thích dài, nhãn không thống nhất.

Prompt B

Nêu role, task, label set, constraint và JSON schema.

Output thường dễ kiểm thử/tích hợp hơn.

Bài 3: biến prompt thành đặc tả công việc có thể kiểm thử.

53. Failure mode 3: Knowledge boundary

- Model có thể không biết tài liệu nội bộ của tổ chức.
- Kiến thức trong pre-training có thể thiếu, sai hoặc không cập nhật.
- Cùng một thuật ngữ nội bộ có thể mang nghĩa khác so với dữ liệu công khai.

Câu hỏi thiết kế

Nếu câu trả lời phải bám "phiên bản SOP hiện hành", nguồn sự thật nằm ở model hay ở kho tài liệu?

Đây là động lực chính dẫn sang semantic search và RAG ở Bài 5–6.

54. Failure mode 4: Context limitation

- Context có giới hạn hữu hạn.
- Input quá dài có thể làm tăng latency/cost.
- Thông tin quan trọng có thể bị lẫn trong lượng context lớn.
- LLM không tự biết phần nào là "nguồn chính thức" nếu ta không thiết kế context/ràng buộc tốt.

Không phải giải pháp

Copy toàn bộ kho tài liệu vào prompt cho mỗi câu hỏi.

55. Failure mode 5: Nondeterminism / consistency

Cùng một input có thể cho các output khác nhau về:

- cách diễn đạt;
- thứ tự lý do;
- mức tự tin;
- đôi khi cả nhãn/kết luận.

Kiểm soát

Dùng structured output, tham số phù hợp, test set, validation và fallback/human review khi cần.

56. Failure mode 6: Latency và Cost

- Model lớn hơn / prompt dài hơn / output dài hơn thường tiêu tốn nhiều tài nguyên hơn.
- Tác vụ real-time có yêu cầu latency khác tác vụ batch.
- Một PoC chạy tốt 20 request chưa nói lên chi phí khi có 100.000 request/ngày.

Câu hỏi nghiệp vụ

Giá trị của một request AI có lớn hơn chi phí + độ trễ + chi phí kiểm soát lỗi không?

57. Failure mode 7: Privacy và dữ liệu nhạy cảm

- Không đưa dữ liệu nhạy cảm vào dịch vụ/model không được cấp phép.
- Cần biết dữ liệu nào model/service nhận được, lưu ở đâu, ai có quyền truy cập.
- Log/prompt có thể chứa mã khách hàng, thông tin nhân sự, cấu hình hạ tầng hoặc dữ liệu nội bộ.

Nguyên tắc

Phân loại dữ liệu trước khi chọn công cụ AI, không làm ngược lại. (Chi tiết ở Bài 2.)

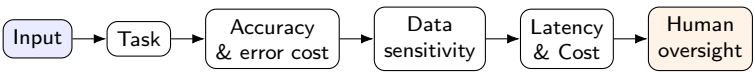
58. Sai 10% không phải lúc nào cũng giống nhau

Tóm tắt	Khuyến nghị	Hành động
Sai câu chữ nhẹ	Bỏ sót cảnh báo quan trọng	Tự thay đổi cấu hình sai
Rủi ro tương đối thấp	Rủi ro trung bình/cao	Rủi ro rất cao

Thông điệp

Độ chính xác phải được đọc cùng **hậu quả của lỗi** và khả năng đảo ngược hành động.

59. Khung quyết định có nên dùng LLM?



Không có một câu trả lời "LLM phù hợp" chỉ dựa trên loại model; phải xét toàn bộ chuỗi quyết định.

60. Bước 1–2: Input và Task

Input là gì?

- text tự do, bảng số, hình ảnh, audio, tài liệu nội bộ?
- dữ liệu có cấu trúc hay phi cấu trúc?

Task thật sự là gì?

- predict, classify, extract, summarize, retrieve, generate hay take action?

Tránh mô tả mơ hồ

"Xây chatbot AI" chưa phải requirement nghiệp vụ đủ tốt.

61. Bước 3: Accuracy và business impact

Hỏi ít nhất bốn câu:

1. Mức đúng cần thiết là bao nhiêu?
2. False positive gây hậu quả gì?
3. False negative gây hậu quả gì?
4. Hallucination có thể được phát hiện trước khi gây tác động không?

Ví dụ

Sai nhãn ticket có thể chỉ làm chậm routing; sai khuyến nghị thay đổi cấu hình có thể gây outage.

62. Bước 4: Data sensitivity

Nhóm	**Câu hỏi trước khi gửi vào AI service**
Public	Có thể công khai?
Internal	Có được phép gửi ra ngoài tổ chức không?
Confidential	Có cần môi trường/model được phê duyệt riêng không?
Personal	Có dữ liệu cá nhân / định danh / quyền riêng tư không?

Chi tiết governance sẽ được tổng hợp ở Bài 2, nhưng privacy phải được "cài" từ Bài 1.

63. Bước 5–6: Latency, Cost và Human oversight

Latency / Cost

- real-time hay batch?
- số request dự kiến?
- output dài bao nhiêu?
- có cache/fallback không?

Human oversight

- AI chỉ hỗ trợ?
- AI chuẩn bị, người duyệt?
- AI được phép tự hành động?

Mặc định an toàn

Rủi ro càng cao, quyền hành động của AI càng cần bị giới hạn và có bước phê duyệt.

64. Decision Challenge: chọn công cụ và giải thích

Với 6 case đầu buổi, mỗi nhóm chọn một phương án và nêu **1 lý do + 1 rủi ro**:

Case	**Chọn gì? Vì sao? Rủi ro?**
Dự báo KPI	
Phân loại ticket	
Tóm tắt báo cáo	
Hỏi sổ tay nội bộ	
Đọc ảnh thiết bị	
Đề xuất xử lý	

65. Tổng kết Bài 1 và cầu nối sang Bài 2

5 ý cần nhớ

1. Bắt đầu từ nghiệp vụ, không từ model.
2. LLM xử lý token trong context và sinh token tiếp theo theo xác suất.
3. Transformer dùng Self-Attention (Q/K/V) và Multi-Head Attention để nắm quan hệ trong context.
4. BERT (hiểu), GPT (sinh), LLaMA (open-weight) phù hợp với các loại nhiệm vụ khác nhau.
5. Quyết định dùng LLM phải xét: Input $\rightarrow$ Task $\rightarrow$ Accuracy $\rightarrow$ Data sensitivity $\rightarrow$ Latency/Cost $\rightarrow$ Human oversight.

Bài 2

Trước khi xây giải pháp: **Responsible AI, Explainable AI, kiểm soát bias, bảo mật dữ liệu khách hàng và khung đánh giá rủi ro AI**.